

S

熟悉面向对象分析与设计工具书

基于 SprintBoot 与 Vue 的商城系统

Litemall

实验报告书

EXPREIMENTAL REPORT

实验成员

崔胤康 胡睿杰 韩方俊

周厚泽 曹子墨

2025/10/10



目 录

第 1 章 实验介绍	4
1.1 实验名称	4
1.2 实验目的	4
1.3 实验成员	4
第 2 章 项目概要	4
2.1 项目名称	4
2.2 项目背景	5
2.3 项目简述	6
2.3.1 后端与数据库	6
2.3.2 前端	8
第 3 章 面向对象核心概念学习	13
3.1 类 (Class)	13
3.2 对象 (Object)	14
3.3 属性 (Attribute)	14
3.4 消息 (Message)	14
3.5 继承 (Inheritance)	15
3.6 聚合 / 组合 (Aggregation / Composition)	15
3.6.1 基本定义	15
3.6.2 语义差异与生命周期管理	16
3.6.3 设计与语义层面意义	16
第 4 章 OOSAD 模型、方法与工具调研	17
4.1 面向对象建模	17
4.1.1 背景与定位	17
4.1.2 用例图 (Use Case Diagram) ^[10]	17

4.1.3 类图 (Class Diagram) ^[11]	18
4.1.4 对象图 (Object Diagram) ^[12]	19
4.1.5 顺序图 (Sequence Diagram) ^[13]	20
4.1.6 四种模型的比较与联系	21
4.2 用例模型	22
4.2.1 概述与基本概念	22
4.2.2 标准化建模技术	23
4.2.3 行业实践与工具支持	24
4.3 分析模型	24
4.3.1 概述与基本概念	24
4.3.2 分析模型的组成元素	25
4.3.3 操作契约 (Operation Contract)	27
4.4 设计模型	27
4.4.1 定义与核心目标	27
4.4.2 设计模型的组成元素	28
4.4.3 设计模式 (Design Patterns)	29
4.4.4 分层架构 (Layered Architecture)	29
4.5 UML 建模工具绘制	30
4.5.1 用例图	30
4.5.2 类图	31
4.5.3 顺序图	32
4.5.4 对象图	32
4.5.5 组件图	34
第 5 章 会议记录	34
5.1 第一次会议记录	34
5.2 第一次会议记录	35
参考文献	37

第 1 章 实验介绍

1.1 实验名称

熟悉面向对象分析与设计(OOSAD)工具

1.2 实验目的

1. 熟悉使用至少一种 UML 建模工具
2. 熟悉使用至少一种数据库建模工具
3. 尝试使用 UML 建模工具及数据库建模工具绘制基本模型

1.3 实验成员

姓名及贡献度	学号
崔胤康（20%）	222024321062038
周厚泽（20%）	222024321062042
曹子墨（20%）	222024321062045
胡睿杰（20%）	222024321062046
韩方俊（20%）	222024321062047

表 1 实验成员表

第 2 章 项目概要

2.1 项目名称

基于 SpringBoot 与 Vue 的商城系统-Litemall

2.2 项目背景

当前，全球电商行业正由“规模扩张”转向“精细化运营”和“数字化普惠”阶段。随着市场竞争日益激烈，中小微企业和初创团队对电商系统的需求也随之聚焦于“轻量化”与“实用性”的平衡。一方面，AI 智能推荐、私域运营已成为提升转化率和用户粘性的行业标配，然而，市面上主流商业解决方案往往价格高昂、部署复杂，这与初创团队低成本、易上手的诉求形成显著矛盾。另一方面，多端协同（如小程序、H5、PC）已是基础需求，而系统部署效率直接影响商家的市场启动速度，迫切需要“开箱即用”的解决方案以抢占商业先机。

痛点	描述	对业务的影响
高昂的初期投入	商业电商系统授权费、部署费高昂，增加了初创团队的资金压力。	阻碍快速市场切入和规模化扩张。
部署与运维困难	复杂的系统架构需要专业的运维团队，提高了人力成本和系统故障风险。	延长项目启动周期，影响商业时效性。
多端协同不力	缺乏原生、高效支持小程序、H5、PC 端等多渠道的能力，导致数据同步和用户体验分散。	限制了流量获取渠道和用户覆盖面。
技术选型壁垒	大部分开源系统技术栈老旧或文档不全，难以二次开发和人才招聘。	限制了系统的长期生命力和定制化能力。

表 2 中小微商家在数字化转型中面临的核心痛点

正是基于这一行业痛点与市场机遇，该项目——开源电商系统 **Lite­mall** 应运而生。**Lite­mall** 采用当前主流、稳定的 **Spring Boot + Vue** 技术栈，其核心优势完美契合了中小微企业的轻量化需求：模块化架构极大降低了系统的部署和学习难度，实现了真正的快速启动；同时，系统原生支持微信小程序、H5 与 PC 端协同，消除了跨平台开发的壁垒，充分满足了中小商家“快速上线、全渠道覆盖”的核心诉求。

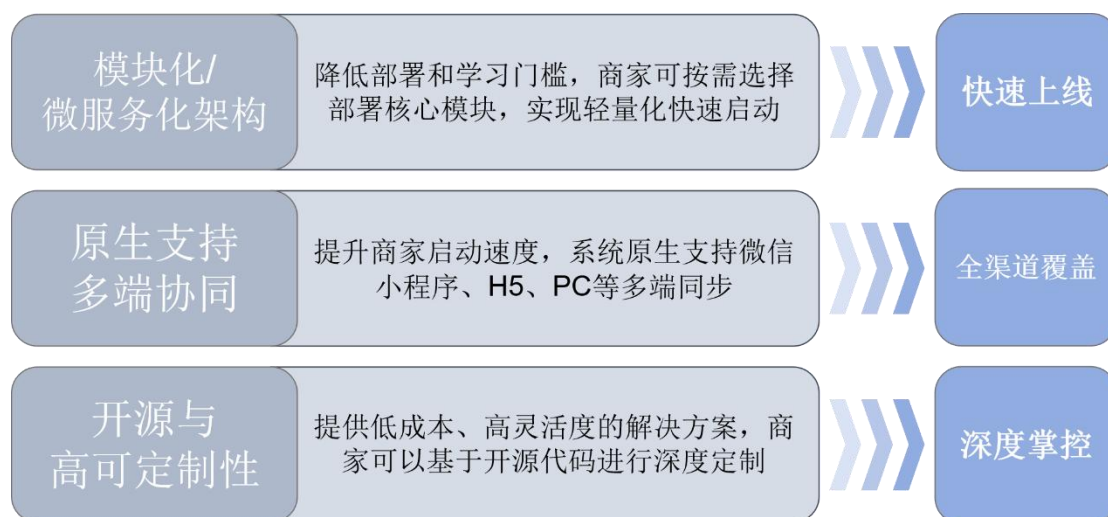


图 1 Litemall优势分析图

综上所述，Litemall 的价值定位在于：为广大小微企业、初创团队及开发者社区提供一个低门槛、高效率、可扩展的电商基础设施。它不仅是搭建电商服务的技术捷径，更是助力中小企业在激烈的市场竞争中，以更低的成本、更高的效率实现数字化转型的有力工具。

2.3 项目简述

本项目主要分为前端+后端两部分

2.3.1 后端与数据库

后端使用 SpringBoot 框架，负责具体功能的包及类有：

- (1) litemall-db：负责调用 MyBatis 自动生成处理数据库的代码
- (2) litemall-core：负责项目的核心服务管理，包括：
 - ① AsyncConfig 类负责数据库操作的异步执行
 - ② CorsConfig 为不同域名的前端共享资源
 - ③ SystemConfig 类将所有前端的后端的配置映射到哈希图上
 - ④ ExpressInfo, Trace 类封装快递查询结果（数据传输对象，DTO）
 - ⑤ GlobalExceptionHandler 类处理全局异常
 - ⑥ JacksonConfig 类用于处理 json 文件使得 SpringBoot 可以接受
 - ⑦ ValidatorConfiguration 类用于验证流入后端的数据格式正确

- 数据库使用 MySQL，具体表的情况如图：

[illegible]

图 2 MySQL表总览图

litemall_ad	3	16 KB	InnoDB	3	广告表
litemall_address	0	16 KB	InnoDB	0	收货地址表
litemall_admin	5	16 KB	InnoDB	3	管理员表
litemall_aftersale	0	16 KB	InnoDB	0	售后表
litemall_brand	1046000	16 KB	InnoDB	49	品牌商表
litemall_cart	0	16 KB	InnoDB	0	购物车商品表
litemall_category	1036004	48 KB	InnoDB	93	类目表
litemall_collect	0	16 KB	InnoDB	0	收藏表
litemall_comment	1014	256 KB	InnoDB	1004	评论表
litemall_coupon	8	16 KB	InnoDB	4	优惠券信息及规则表
litemall_coupon_user	0	16 KB	InnoDB	0	优惠券用户使用表
litemall_feedback	0	16 KB	InnoDB	0	意见反馈表
litemall_footprint	2	16 KB	InnoDB	2	用户浏览足迹表
litemall_goods	1181000	2576 KB	InnoDB	231	商品基本信息表
litemall_goods_attribute	876	112 KB	InnoDB	876	商品参数表
litemall_goods_product	244	64 KB	InnoDB	244	商品货品表
litemall_goods_specification	243	16 KB	InnoDB	243	商品规格表
litemall_groupon	0	16 KB	InnoDB	0	团购活动表
litemall_groupon_rules	3	16 KB	InnoDB	3	团购规则表
litemall_issue	4	16 KB	InnoDB	4	常见问题表
litemall_keyword	7	16 KB	InnoDB	7	关键字表
litemall_log	44	16 KB	InnoDB	0	操作日志表
litemall_notice	0	16 KB	InnoDB	0	通知表
litemall_notice_admin	0	16 KB	InnoDB	0	通知管理员表
litemall_order	0	16 KB	InnoDB	0	订单表
litemall_order_goods	0	16 KB	InnoDB	0	订单商品表
litemall_permission	31	16 KB	InnoDB	31	权限表
litemall_region	3231	176 KB	InnoDB	3231	行政区域表
litemall_role	3	16 KB	InnoDB	3	角色表
litemall_search_history	0	16 KB	InnoDB	0	搜索历史表
litemall_storage	0	16 KB	InnoDB	0	文件存储表
litemall_system	18	16 KB	InnoDB	18	系统配置表
litemall_topic	314	16 KB	InnoDB	20	专题表
litemall_user	1	16 KB	InnoDB	0	用户表

图 3 MySQL表目录图

2.3.2 前端

前端分为微信小程序与其他前端，由于两个方面极其相似，这里仅以微信小程序为例

针对电商核心场景（如商品浏览、下单、支付），前端交互采用“三步内完成核心操作”的设计原则：

- 1) **商品选购环节：**支持“点击商品卡片直接查看详情”“下拉加载更多商品”，避免分页跳转；详情页内集成“加入购物车”“立即购买”双按钮，且点击后无需跳转新页，通过弹窗完成规格选择（如尺寸、颜色），减少页面切换带来的体验中断。

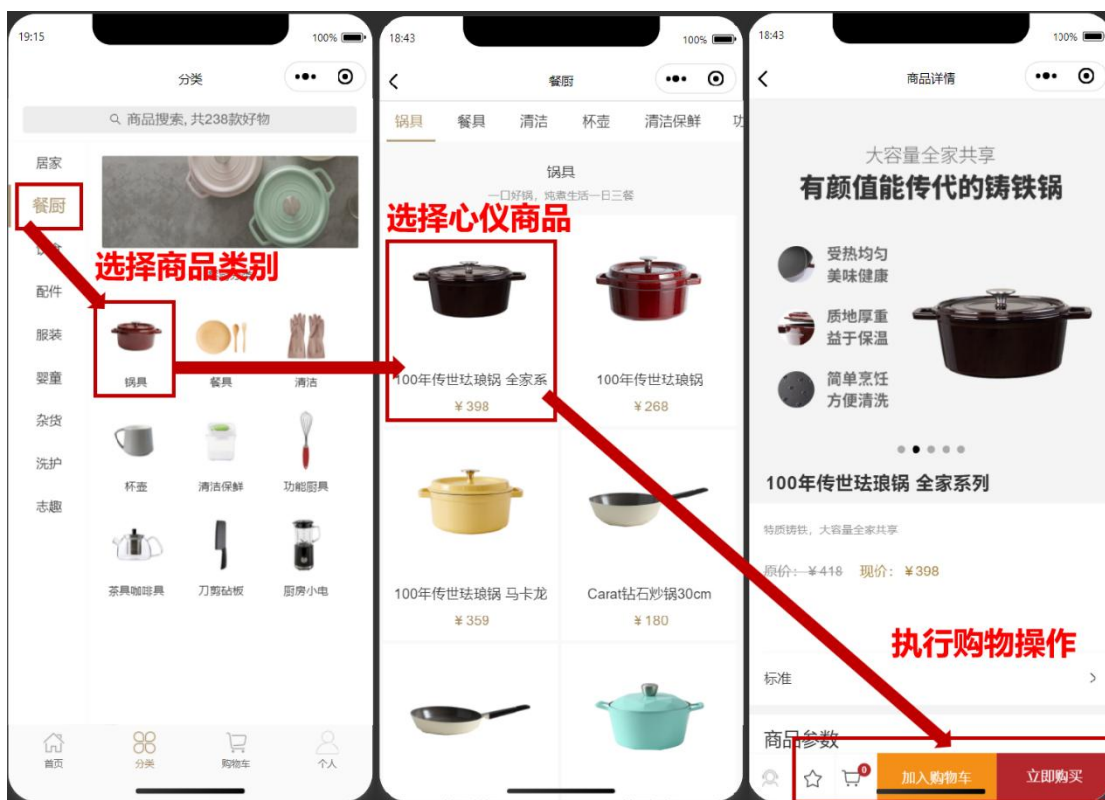


图 4 商品挑选操作示意图

- 2) **下单支付环节：**购物车结算时自动填充默认收货地址，支持“一键修改地址”弹窗；支付方式选择页采用“标签式切换”，无需刷新页面即可切换微信、支付宝等支付渠道，支付结果页提供“返回首页”“查看订单”明确指引，避免用户迷路。

为避免用户因“操作无反馈”产生困惑，前端设计了多维度的即时反馈：

- 1) **加载状态反馈：**商品列表、订单列表等异步加载场景，采用“骨架屏”替代传统加载动画，既告知用户“正在加载”，又通过占位布局提前呈现页面结构，降低等待焦虑。
- 2) **操作结果反馈：**加入购物车成功时，商品卡片会有“飞入购物车”的微动画；表单提交（如修改收货地址）错误时，字段下方会实时显示红色提示文字（如“手机号格式不正确”），而非跳转错误页。

- 3) **状态提示可视化：**订单状态（待付款、待发货、待收货）通过不同颜色的标签区分（红色“待付款”、蓝色“待发货”、绿色“待收货”），配合进度条展示订单流程（如“下单→付款→发货→收货”），让用户直观了解订单进度。

litemall 前端采用响应式设计，适配手机、平板、电脑等不同尺寸的设备，核心优化点包括：

- 1) **移动端优先：**针对手机用户，优化页面布局（如商品列表从“4 列网格”变为“2 列网格”）、放大点击区域（按钮最小尺寸不小于 44px×44px）、隐藏非核心功能（如电脑端的“侧边栏分类”在移动端转为“顶部下拉菜单”）。
- 2) **平板 / 电脑端适配：**在大尺寸屏幕上，增加“商品对比”“批量操作订单”等功能，利用多列布局提升信息密度（如订单列表显示“订单号、商品、金额、状态、操作”完整信息，无需折叠）。
- 3) **横竖屏适配：**移动端支持横竖屏自动调整布局，如横屏时商品详情页改为“左侧图片、右侧信息”的分栏布局，提升大屏浏览体验。

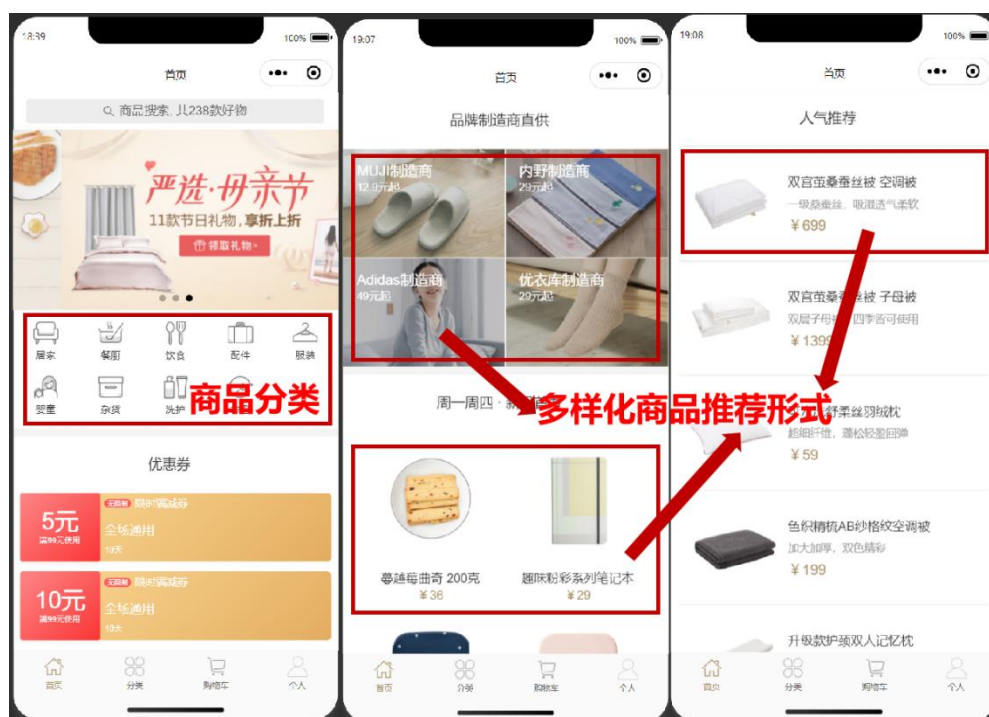


图 5 小程序端主页示意图

litemall 前端功能模块围绕“用户购物全流程”设计，涵盖“商品浏览 - 选购 - 下单 - 支付 - 售后”全链路，同时支持“用户中心 - 系统设置”等辅助功能，具体模块设计如下：

- 1) **商品分类：**支持“一级分类 + 二级分类”层级展示，分类页可按“销量、价格、好评率”筛选排序，配合“分类图标 + 名称”的可视化设计，帮助用户快速定位品类（如“电子产品→手机→智能手机”）。
- 2) **商品搜索：**支持“关键词搜索 + 历史搜索记录 + 热门搜索推荐”，输入关键词时实时联想（如输入“手机”，联想“手机壳、手机膜、智能手机”），搜索结果页支持“筛选（品牌、价格区间、销量）”和“排序”，提升搜索效率。
- 3) **商品详情：**详情页采用“顶部轮播图（支持缩放查看细节）→商品名称 + 价格 + 优惠信息→规格选择（弹窗）→商品详情图文→用户评价→相关推荐”的布局，核心信息（价格、优惠、库存）固定在顶部，避免用户滚动时丢失关键信息。
- 4) **购物车管理：**支持“单选 / 全选商品”、“修改商品数量（± 按钮 + 直接输入）”、“删除商品”、“移入收藏夹”等功能，商品卡片显示“商品图片、名称、规格、单价、小计”，底部实时计算“选中商品总价”，并提示“满减优惠”（如“满 200 减 30，还差 50 元可享优惠”）。
- 5) **购物车同步：**支持“登录 / 未登录状态同步”选项，未登录时购物车数据存储在本地，登录后自动合并到账号中，避免用户切换设备或登录后丢失选购商品。
- 6) **订单创建：**从购物车结算后，自动生成订单，支持“修改收货地址”、“选择优惠券”、“备注订单信息”，确认订单页清晰展示“商品清单、优惠金额、实

付金额”，避免用户因信息不透明放弃下单；

- 7) **支付功能：**集成主流支付渠道（微信支付、支付宝），支持“跳转第三方支付页面”或“内嵌支付组件”，支付完成后自动跳转“支付结果页”，并通过弹窗提示“支付成功”，同时发送短信通知；
- 8) **订单管理：**支持“按订单状态筛选（待付款、待发货、待收货、已完成、已取消）”、“查看订单详情”、“取消订单（待付款状态）”、“申请退款（待发货 / 待收货状态）”、“确认收货”，订单详情页展示“物流信息（实时跟踪快递进度）”、“商品信息”“支付信息”，方便用户追溯订单全流程。
- 9) **个人信息管理：**支持“修改头像、昵称、手机号”及“绑定第三方账号（微信、QQ）”，信息修改实时生效，无需跳转审核页；

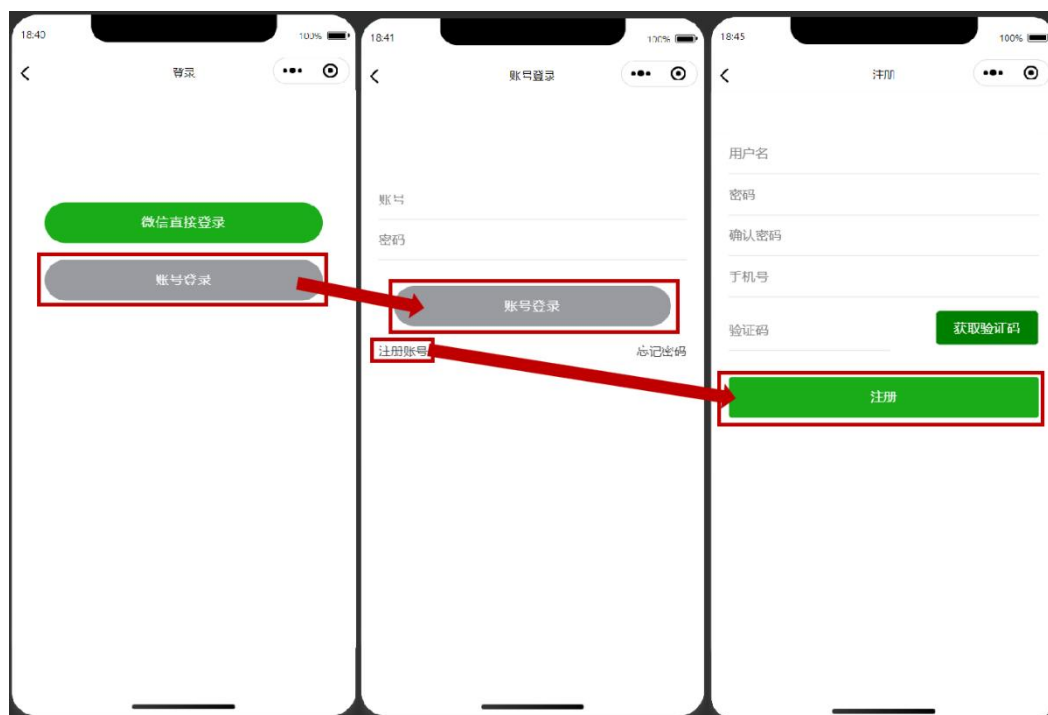


图 6 小程序登录功能示意图

- 10) **售后服务：**集成“退款申请（填写退款原因、上传凭证）”、“售后进度查询”、“售后沟通（与商家在线聊天）”等服务选项，退款状态变更时通过系统消息通知用户。

- 11) **收藏与足迹**：支持“收藏商品（点击红心添加，再次点击取消）”、“查看浏览足迹（按时间排序，支持删除单个 / 清空）”等功能，收藏商品降价时发送“降价提醒”，提升用户复购概率。
- 12) **基础设置**：支持“开启 / 关闭消息通知（订单、活动、售后）”“清除缓存”“切换语言（中文 / 英文）”，设置项采用“开关 + 说明文字”的设计，降低操作难度。
- 13) **安全设置**：提供“修改密码（原密码验证 + 新密码两次输入）”、“设置支付密码”、“绑定手机验证”等操作，敏感操作时通过“短信验证码”验证，保障账号安全。

第 3 章 面向对象核心概念学习

3.1 类（Class）

在面向对象方法中，类（Class）是对一组具有相似属性（状态）和行为（方法或操作）的对象的抽象与定义，类为对象提供了结构与行为的模板，是对象的蓝图。类定义了对对象应具备的属性与方法，但本身并不占用实例空间，只有在实例化为对象时才会生成对应的内存与状态。

正如 Sabah Al-Fedaghi 在《Classes in Object-Oriented Modeling (UML): Further Understanding and Abstraction》^[1]（2021）中所指出的：

“A class is a blueprint for creating objects; it defines the attributes and behaviors that characterize the objects of that class.”（类是创建对象的蓝图，它定义了该类对象所具有的属性与行为）。

此外，Wegner（1987）在《What is Object-Oriented Programming?》^[2]中亦指出：“A class is an extensible code template for object creation, providing initial values for state and implementations of behavior.”（类是用于创建对象的可扩展代码模板，提供状态的初始值与行为的实现）。

3.2 对象 (Object)

在面向对象方法中，对象是类的具体实例，它体现了该类所定义的属性与行为^[3]。对象是具有状态、行为和标识的实体，由类模板创建^[4]。对象包含类定义的属性（状态）和值（具体数据），并能够执行类所定义的方法（行为 / 操作）。

- 对象是系统中的“实体”，可以相互交互，通过发送和接收消息完成任务。
- 每个对象具有唯一标识（Identity），即便两个对象的状态相同，它们仍被视为不同的实例。
- 对象是面向对象程序运行时的基本单位，而类则是对象的蓝图

举例：在图书馆管理系统中，Book 类定义了书籍的属性与操作，而某一本特定的《数据结构与算法》书就是 Book 类的一个对象，它有具体的标题、作者和 ISBN。

3.3 属性 (Attribute)

在面向对象方法中，属性（Attribute）是类的一个命名特征，用来描述该类实例可以持有的一类取值^[5]。属性与方法共同定义了对应的结构，其中属性体现了对象的状态，而方法体现其行为^[6]。

- 属性是类定义的一部分，表示该类的对象应当具有哪些数据成员。
- 每个对象都拥有这些属性的具体取值，从而区分不同对象的状态。
- 属性与方法(行为)共同定义了类的结构和职责：属性代表“静态特征”，方法代表“动态行为”。

例如，在 Book 类中，属性可以包括 title（书名）、author（作者）、ISBN（书号）等。每个对象（具体的书籍）都会拥有这些属性的具体值。

3.4 消息 (Message)

在面向对象方法中，消息（Message）是对象之间传递信息和请求操作的手段，用于触发对象的方法执行，从而实现对象间的交互。对象通过发送消息互相通信，请求对方提供服务，消息指定要调用的操作及所需参数^[6]。也就是，消息是发送给对象的服务请求规范，它可能触发对象的某个操作^[5]。

例如，图书馆系统中，Member 对象向 Book 对象发送 borrow() 消息，请

求借阅某本书，Book 对象接收到消息后执行方法更新状态。

3.5 继承 (Inheritance)

在面向对象方法中，继承 (Inheritance) 是一种机制，允许从已有类派生出新类，从而共享其属性与操作^[6]。

通过继承，可以实现代码复用、系统扩展与结构化设计。

- 子类继承父类的属性 (attributes) 和方法 (operations)，并可以对其进行扩展或重写 (override) ^[5]。
- 继承体现了面向对象的层次结构 (hierarchical structure) 与“is-a”关系，即子类是父类的一种特殊化形式。
- 继承增强了软件的可复用性与可维护性，是面向对象四大特性（封装、继承、多态、抽象）之一。
- 继承允许通过对现有类的特化逐步定义新的行为^[7]。

例如，Student 类可以继承 Person 类的属性（如姓名、年龄）和方法（如显示信息），并新增特有的属性（如学号）和方法（如选课）。

3.6 聚合 / 组合 (Aggregation / Composition)

3.6.1 基本定义

在面向对象分析与设计中，聚合 (Aggregation) 与组合 (Composition) 都是用于表示类与类之间的“整体—部分 (whole-part)”关系的机制，它们主要区别在于所有权的强度以及生命周期的依赖程度^[6]。

它们属于对象之间的关联关系 (association relationship) 的一种特殊形式^[5]，用于表达一个对象（整体）如何包含或由另一个对象（部分）组成。

举例：一个“学校 (School)”包含多个“教师 (Teacher)”对象是**聚合**关系（教师可脱离学校存在），而“人 (Person)”与“心脏 (Heart)”之间的关系是**组合**（心脏不能脱离人独立存在）。

3.6.2 语义差异与生命周期管理

特征	聚合（Aggregation）	组合（Composition）
所属关系	弱拥有（部分可独立存在）	强拥有（部分依附整体）
生命周期	整体与部分生命周期独立	部分随整体创建与销毁
UML 表示	空心菱形箭头（◇）	实心菱形箭头（◆）
示例	学校 - 教师，图书馆 - 书籍	人 - 心脏，房屋 - 房间

表 3 聚合与组合的特征差异表

这种区分最早在 UML 的关系模型中被明确化，强调生命周期依赖是两者最本质的区别^[14]。

3.6.3 设计与语义层面意义

聚合与组合体现了面向对象中对象之间的结构组织与责任划分。相比继承表达的“is-a”关系，聚合与组合体现的是“has-a”关系。

在面向对象设计（OOD）中，它们被广泛用于：

- 表达复杂系统的层级结构；
- 实现模块化与封装；
- 管理对象间的生命周期与依赖关系；
- 支撑“组合优于继承（composition over inheritance）”的设计原则。

GoF（《Design Patterns》）指出：“Favor object composition over class inheritance”（优先选择对象组合而不是类继承），因为组合通过对象引用实现行为复用，耦合更低、灵活性更高^[15]。

第 4 章 OOSAD 模型、方法与工具调研

4.1 面向对象建模

4.1.1 背景与定位

在面向对象的软件设计与需求分析过程中，用图形化的模型来描述系统的结构与行为，是一种常见且有效的方法。UML（统一建模语言）为此提供了一套标准图形符号与语义解释的语言^[8]。常见的模型包括用例图（Use Case Diagram）、类图（Class Diagram）、对象图（Object Diagram）、顺序图（Sequence Diagram）等^[9]。

4.1.2 用例图（Use Case Diagram）^[10]

用例图用于表示系统与外部实体之间的交互关系，主要聚焦于系统提供的功能（用例）和这些功能与谁（actor）有关联。它是从“谁要做什么”这个层面来刻画系统的边界与功能视图。

使用场景 / 作用

- 在需求调研或需求梳理阶段，用来和业务方、用户沟通系统边界和功能需求；
- 在需求说明文档中作为功能概览，帮助开发人员、测试人员快速理解系统作用域；
- 用来识别系统的角色 / 参与者（Actor）和功能用例（Use Case），为后续更详细设计做铺垫；
- 作为确认系统范围（哪些功能属于系统，哪些在外部系统等）的一种“地图”视图。

主要图形元素

元素	符号/表示	含义/说明
Actor（参与者）	小人图标	表示外部用户或系统
Use Case（用例）	椭圆形	系统对外提供的功能

元素	符号/表示	含义/说明
Association（关联）	实线连接	Actor 与 Use Case 的交互
«include» / «extend»	虚线 + 标签	用例间的包含或扩展关系

表 4 用例图主要图形元素图

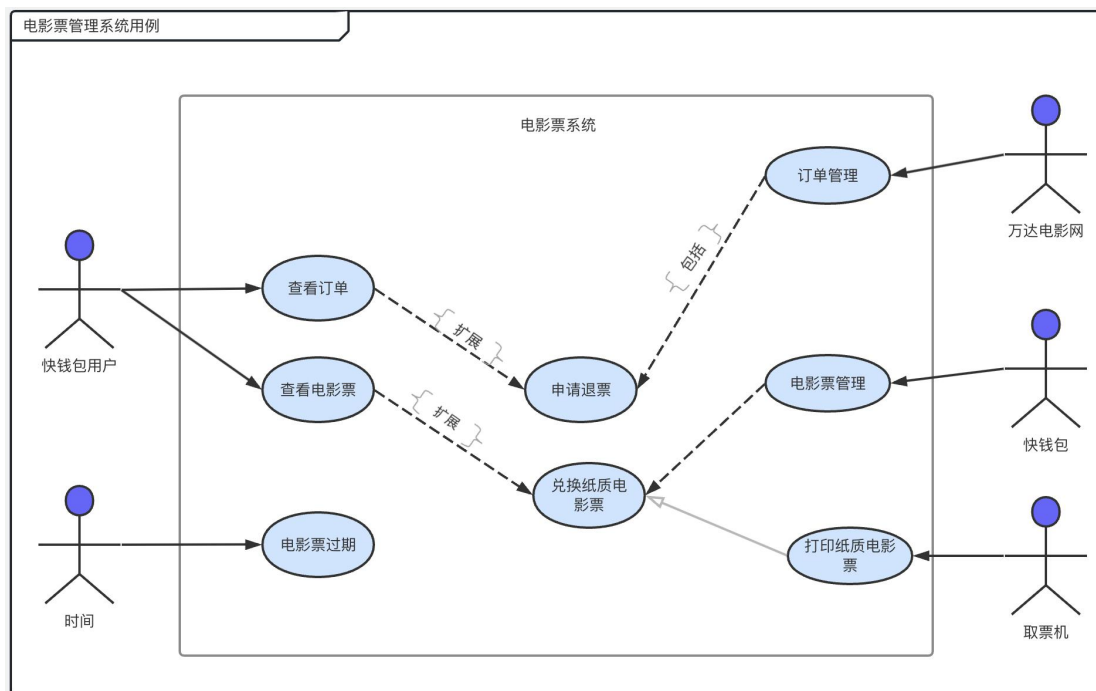


图 7 用例图示例

4.1.3 类图（Class Diagram）^[11]

类图（Class Diagram）是一种静态结构图，用来表示系统中类（class）、接口（interface）及它们之间的关系（关联、继承、依赖、组合、聚合等）。它主要描述系统数据结构、对象属性与操作的静态视图。类图是面向对象设计中最基本也最核心的图。

使用场景 / 作用

- 在设计阶段，用类图来规划系统的结构：有哪些类、每个类的属性与方法，以及类之间如何关联；
- 作为类设计和代码实现的蓝图，指导开发人员编写对应的类；
- 用于数据库与对象模型之间的映射（ORM / 持久化设计）；
- 在维护阶段，用来理解系统结构、定位模块间依赖。

主要图形元素

元素	符号/表示	含义/说明
Class（类）	矩形三层结构	显示类名、属性、方法
Association（关联）	实线	类之间的关系
Inheritance（继承）	空心箭头	子类继承父类
Aggregation / Composition	空心或实心菱形	表示聚合或组合关系

表 5 类图主要图形元素表

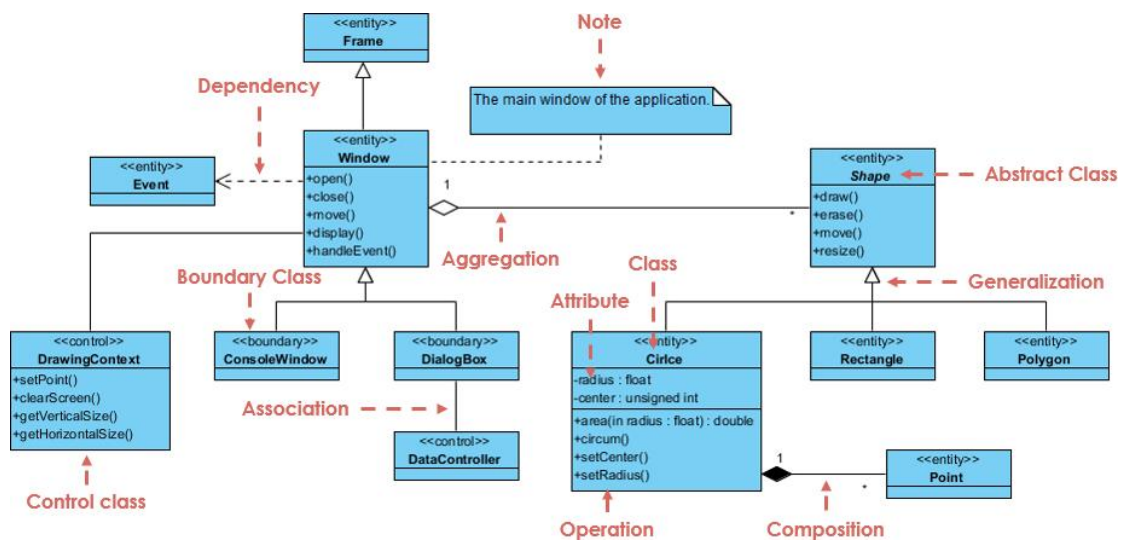


图 8 类图介绍图

4.1.4 对象图（Object Diagram）^[12]

对象图是一种静态图，不同于类图，它描述系统在某个时间点上具体对象实例（objects）之间的关系和状态。通俗地说，对象图是类图在某个瞬时的一个“快照”。

- 用于说明或验证类图设计是否合理；
- 在需求、测试或文档中，以具体情景展示对象间的状态或关系；
- 在教学或设计评审中，用实例图辅助理解类图更抽象的关系。

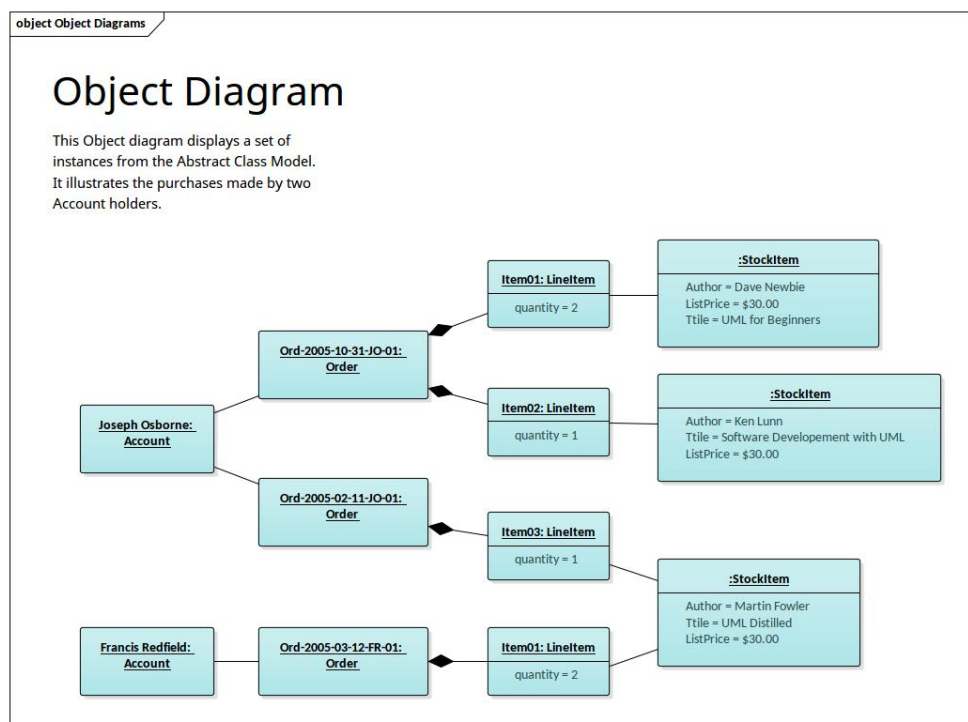


图 9 对象图示例

4.1.5 顺序图（Sequence Diagram）^[13]

顺序图（Sequence Diagram）属于交互图，用于描述对象或实例之间按时间顺序发送消息（请求/响应）的交互过程。它将时序维度贯穿图中，强调对象间“谁先调用谁、何时调用”的顺序关系。

使用场景 / 作用

- 从某个用例或场景出发，描述系统内部对象之间交互的细节；
- 在设计阶段验证或优化对象之间的通信顺序；
- 确定责任分配，分析调用链的执行过程；
- 用于性能分析、协议设计、调试与验证。

主要图形元素

元素	符号/表示	含义/说明
Lifeline（生命线）	对象名称 + 垂直虚线	表示对象在交互中的生存时间线
Activation（激活）	矩形条	表示对象执行操作的时间段
Message（消息）	箭头	表示调用请求
Return Message（返回消息）	虚线箭头	表示返回结果

元素	符号/表示	含义/说明
Self-message（自调用）	自身循环箭头	表示对象自身方法调用
Combined Fragments	alt、loop 等框	表示条件、循环、并行等结构

表 6 顺序图主要图形元素表

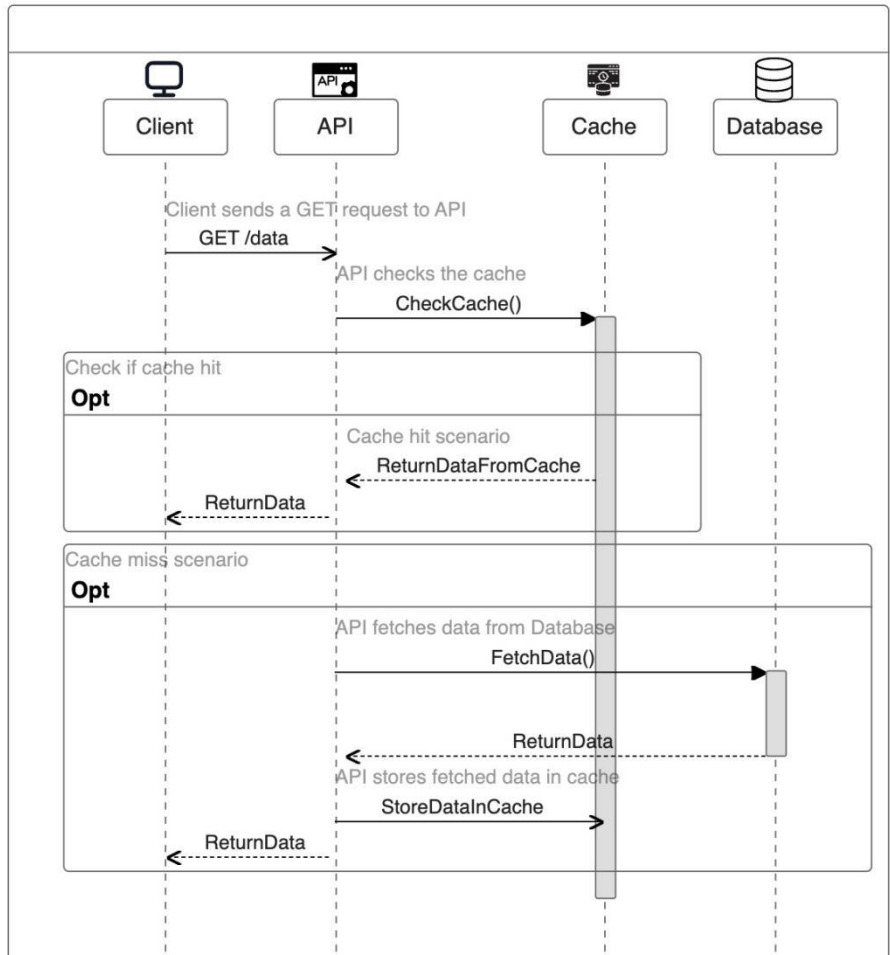


图 10 顺序图示例

4.1.6 四种模型的比较与联系

图类型	主要视角	静态/动态	对象层次	是否有时序	典型阶段
用例图	功能与参与者	静态	抽象角色	无	需求分析
类图	结构设计	静态	类	无	系统设计
对象图	实例快照	静态	对象	无	验证
顺序图	消息交互	动态	对象	有	详细设计

表 7 模型比较表

4.2 用例模型

4.2.1 概述与基本概念

1. 定义与发展历程

用例模型（Use Case Model）是 Ivar Jacobson 博士在 1987 年提出的面向对象需求分析方法^[16]。Jacobson 等人(1992)将其明确定义为："一种通过描述外部参与者与系统交互实现目标来表达功能需求的建模方法，它界定系统边界而忽略内部实现细节"^[16]

该方法经历了三个演进阶段：

初创期（1987-1994）：Ericsson 等电信企业首次应用^[15]

标准化期（1995-2003）：被纳入 UML 1.x 标准^[17]

敏捷适应期（2004 至今）：与 Scrum 等敏捷方法融合^[18]

2. 核心组成元素

参与者（Actor）

Bittner 和 Spence(2002)的经典定义："与系统产生价值交换的任何外部实体（人、系统或设备）"^[19]

主要类型：

- 主要参与者：直接获取核心价值（如电商买家）^[20]
- 支持参与者：提供辅助服务（如支付系统）^[21]
- 离参与者：被动接收信息（如打印机）

用例（Use Case）

Cockburn(2000)提出的关键特征^[20]：

- 目标导向性：必须实现明确目标
- 原子性：要么完全执行，要么完全不执行
- 交互性：包含系统与参与者的双向交互

- 独立性：最小化依赖其他用例
- 价值性：必须为至少一个参与者创造价值

3. 与其他模型的关系

比较维度	用例模型	用户故事	业务流程模型
抽象级别	系统级	特性级	组织级
详细程度	中-高	低	高
适用阶段	需求分析	敏捷迭代	业务分析
表现形式	UML 图+文本	卡片+验收标准	BPMN 图

表 8 模型比较表

4.2.2 标准化建模技术

1. 用例图语法规范

核心元素（基于 UML 2.5 标准^[3]）：

元素	表示方法	语法规则	应用示例
参与者	人形图标	名词大写	Customer
用例	椭圆	“动词+名词”短语	Place Order
包含关系	<<include>>虚线箭头	必须执行	登录→验证
扩展关系	<<extend>>虚线箭头	条件执行	结账→用券

表 9 用例语法图

2. 分层建模方法

Kulak 和 Guiney(2004)的三层体系^[22]：

- 业务层（白盒）：描述组织级流程（如"订单履约"），参与者为部门角色^[23]。
- 系统层(黑盒)：定义软件功能（如"提交订单"），参与者为终端用户^[24]。
- 组件层（灰盒）模块实现（如"计算税费"）参与者为其他组件^[25]。

4.2.3 行业实践与工具支持

1. 应用模式对比

项目类型	用例粒度	文档详略	典型工具
敏捷小项目	粗粒度	用户故事	Jira + Confluence ^[26]
企业系统	多层次	详细规约	Enterprise Architect ^[27]

表 10 应用模式对比表

2. 最佳实践准则

- 80/20 法则：优先建模 20%核心用例^[28]
- 目标级校验：

用户目标级：保留独立用例

子功能级：降级为包含关系^[29]
- 术语一致性：建立领域词典^[30]

4.3 分析模型

4.3.1 概述与基本概念

1. 定义与目的

分析模型（Analysis Model）是面向对象软件开发过程中的关键制品，Rumbaugh 等人（1991）将其定义为："对问题域的抽象表示，专注于捕获系统应该做什么（what）而非如何实现（how）"^[31]。根据 Larman（2004）的解释，分析模型的主要目的是^[32]：

- 建立精确的需求理解：通过面向对象的视角重新组织用例需求

- 识别关键领域概念：发现业务实体及其相互关系
- 构建系统静态结构：确立未来的软件对象框架
- 规划动态交互流程：描述对象间的协作方式

2. 与相邻制品的关系

Booch（1994）特别强调："分析模型是从业务需求到技术方案的桥梁，它避免了过早考虑实现约束，使开发者能够专注于业务本质。"

下表展示了分析模型与相关制品的对比关系：

维度	用例模型	分析模型	设计模型
关注点	系统功能（用户视角）	概念结构（业务视角）	技术方案（实现视角）
抽象层次	黑盒	灰盒	白盒
变化频率	中	较高	低
主要元素	参与者、用例	概念类、系统操作	设计类、接口

表 11 分析模型与相关制品的对比表

4.3.2 分析模型的组成元素

1. 概念类图（Conceptual Class Diagram）

- 基本构成：Fowler（1997）提出概念类图的三个核心要素^[33]
 - **概念类（Conceptual Class）**：表示业务领域中的重要概念，命名采用业务术语（如"订单"、"发票"），不包含实现细节（如 get/set 方法）。
 - **关联（Association）**：类之间的业务关系，需标注多重性（如 1 对多，0..），命名关联方向（如"客户"→"提交"→"订单"）。
 - **属性（Attribute）**：类的业务特征，包含关键属性（通常<5 个），数据类型使用业务语言（如"金额"而非"float"）。
- 建模示例

图 11 概念类图

2. 系统顺序图 (System Sequence Diagram)

● 元素规范

George 等人 (2007) 定义的 SSD 核心元素^[34]:

- 系统作为黑盒: 用方框图表示, 标注"<<system>>"
- 参与者实例: 用例中的主要参与者
- 系统事件 (System Event):
 - 参与者发起的操作
 - 格式: 操作名(参数):返回类型
 - 示例: submitOrder(items: List<Product>): ConfirmationNumber
- 4. 交互约束:
 - 前置条件: [pre] 库存充足
 - 后置条件: [post] 订单状态 = 已确认

● 示例图示

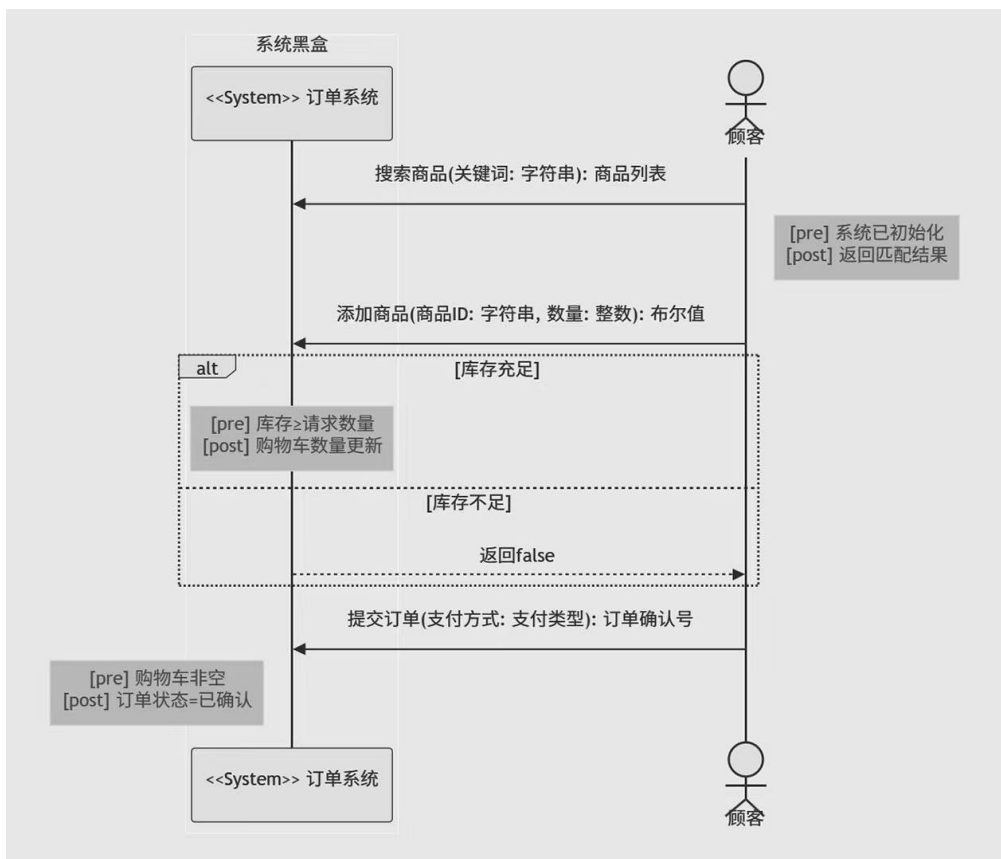


图 12 分析模型系统顺序图示例图示

4.3.3 操作契约（Operation Contract）

设计原则

Wirfs-Brock（2002）的契约设计准则^[35]:

1. 每个契约对应一个系统事件
2. 后置条件使用声明式而非过程式描述
3. 异常情况覆盖所有用例分支
4. 保持契约与技术无关（不涉及数据库操作等）

4.4 设计模型

设计模型是软件开发过程中从需求分析到实现的桥梁，它将分析模型的业务概念转化为可行的技术解决方案^[32]。设计模型不仅要满足功能需求，还需考虑非功能性需求（如性能、可扩展性、安全性等）^[35]

4.4.1 定义与核心目标

1. 定义

设计模型是对计算机系统实现细节的具体描述，它基于分析模型，并加入技术决策（如数据库选择、网络协议、编程语言等）^[37]。

2. 核心目标

- 从业务概念到技术实现：将分析阶段抽象的"做什么"转化为设计阶段的"怎么做"^[32]。
- 架构设计：确定系统整体结构（如分层架构、微服务架构）^[30]。
- 详细设计：定义类的属性和方法、关系、接口、算法等^[14]。
- 优化设计：考虑性能、可扩展性、安全性、可维护性等非功能需求^[38]。

4.4.2 设计模型的组成元素

1. 设计类图（Design Class Diagram）

相较于分析阶段的概念类图，设计类图增加了实现细节^[6]：

设计类（Design Class）

包含完整的方法签名（如 `public void updateOrder(int orderID)`）^[39]

访问控制修饰符（`public`、`private`、`protected`）^[40]

依赖注入（DI）、工厂模式、单例模式等^[41]

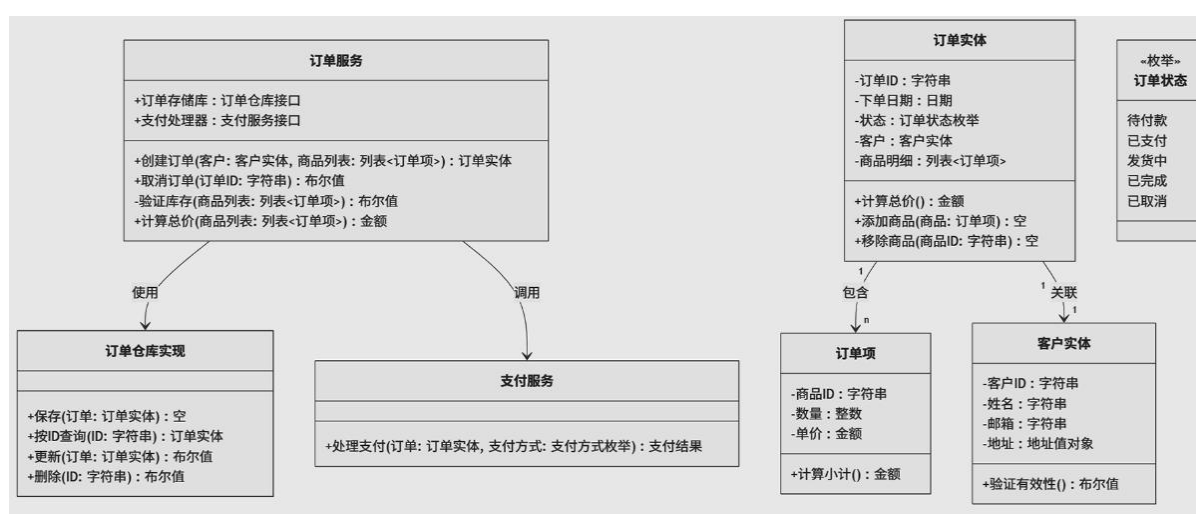


图 13 设计类图

2. 交互图（Interaction Diagram）

➤ 序列图（Sequence Diagram）

描述具体方法调用的顺序，包含^[5]：控制流（如条件分支`alt`、循环`loop`）、数据库访问、外部 API 调用。

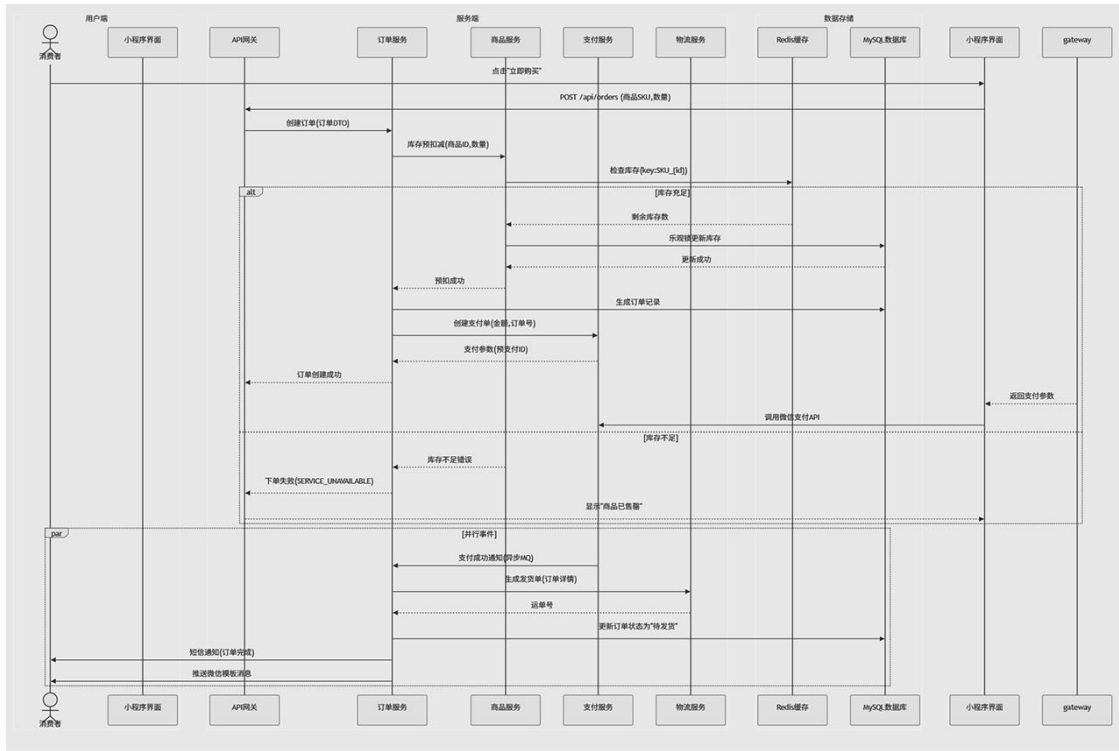


图 14 序列图

➤ 通信图（Communication Diagram）

强调对象间的关系（如消息流）而非时间顺序^[16]。

4.4.3 设计模式（Design Patterns）

在设计中，一些常见问题的标准解决方案^[42]：

分类	常见模式	作用
创建型	Singleton, Factory, Builder	管理对象创建 ^[14]
结构型	Adapter, Decorator, Composite	组织类与对象的关系 ^[30]
行为型	Observer, Strategy, Command	定义对象间的交互 ^[37]

表 12 常见设计模式及作用表

4.4.4 分层架构（Layered Architecture）

典型的软件分层^[38]：

1. 表现层（UI/API Layer）：处理 HTTP 请求、REST/gRPC 交互。
2. 业务逻辑层（Service Layer）：核心业务规则（如库存检查）。

3. 数据访问层(DAO/Repository Layer)：数据库 CRUD 操作(SQL/NoSQL)。
4. 基础设施层 (Infrastructure Layer)：外部服务 (支付、邮件通知)。

4.5 UML 建模工具绘制

针对所选项目，我们实验组所有成员分别使用 StarUML 及 Rational Rose 等绘图软件绘制了用例图、类图、顺序图、对象图以及组件图。

4.5.1 用例图

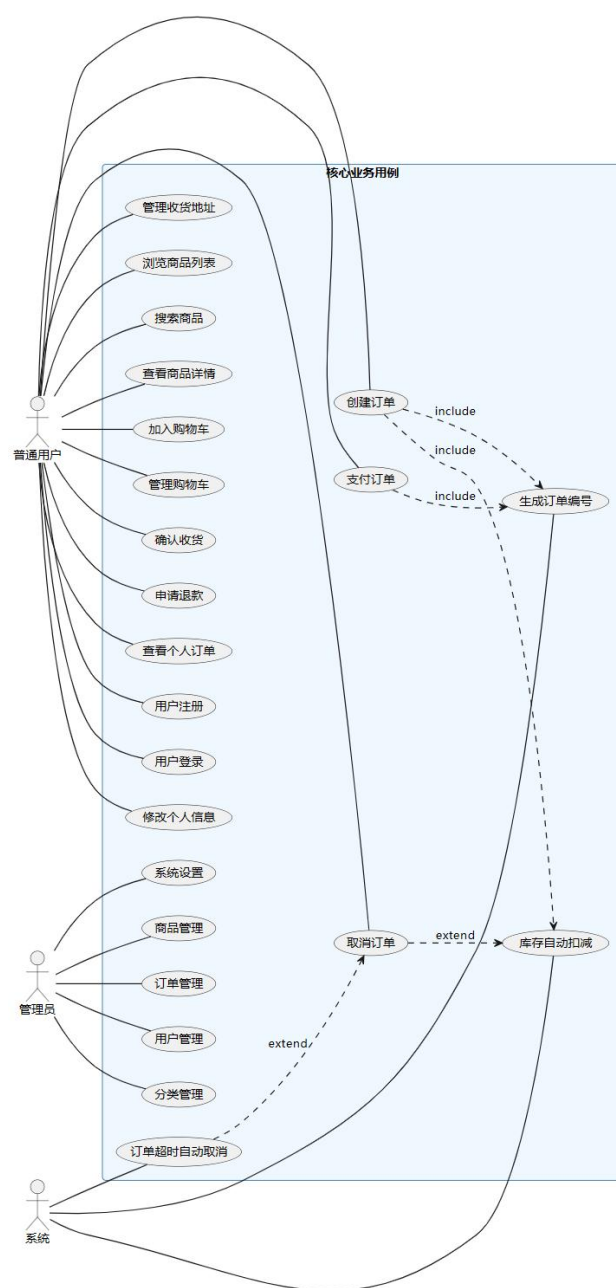


图 15 小组成员绘制用例图

4.5.2 类图

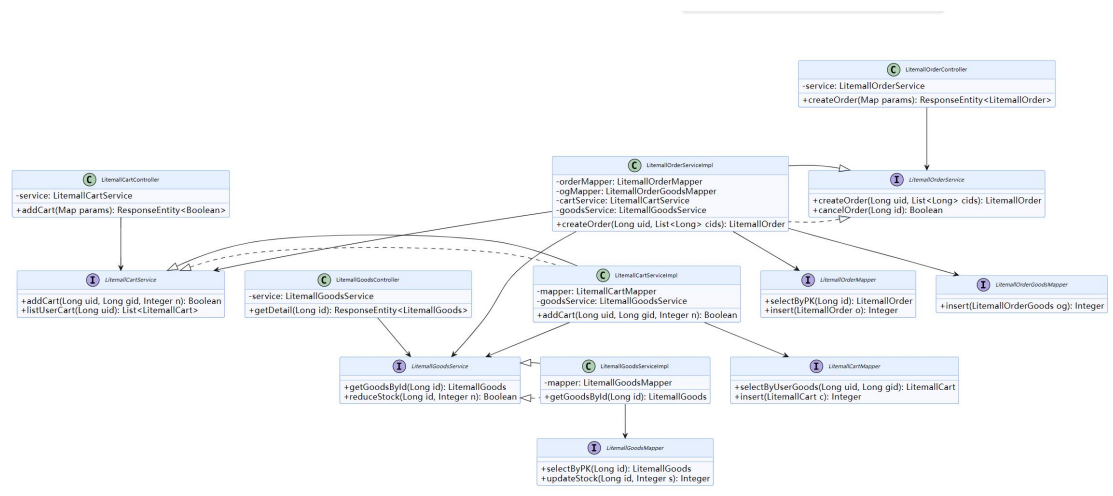


图 16 小组成员绘制用例图一

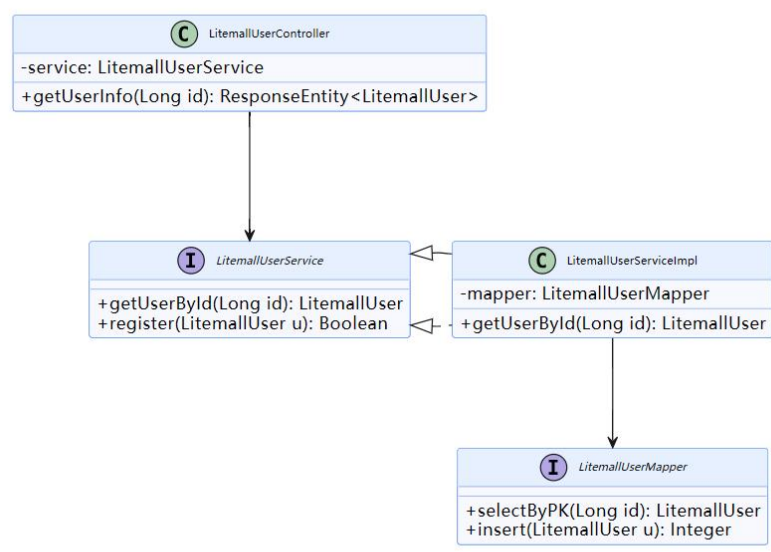


图 17 小组成员绘制用例图二

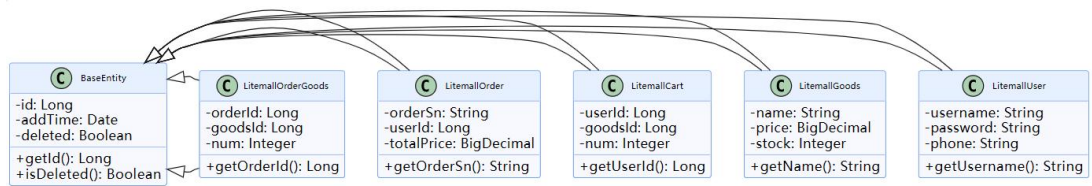


图 18 小组成员绘制用例图三

4.5.3 顺序图

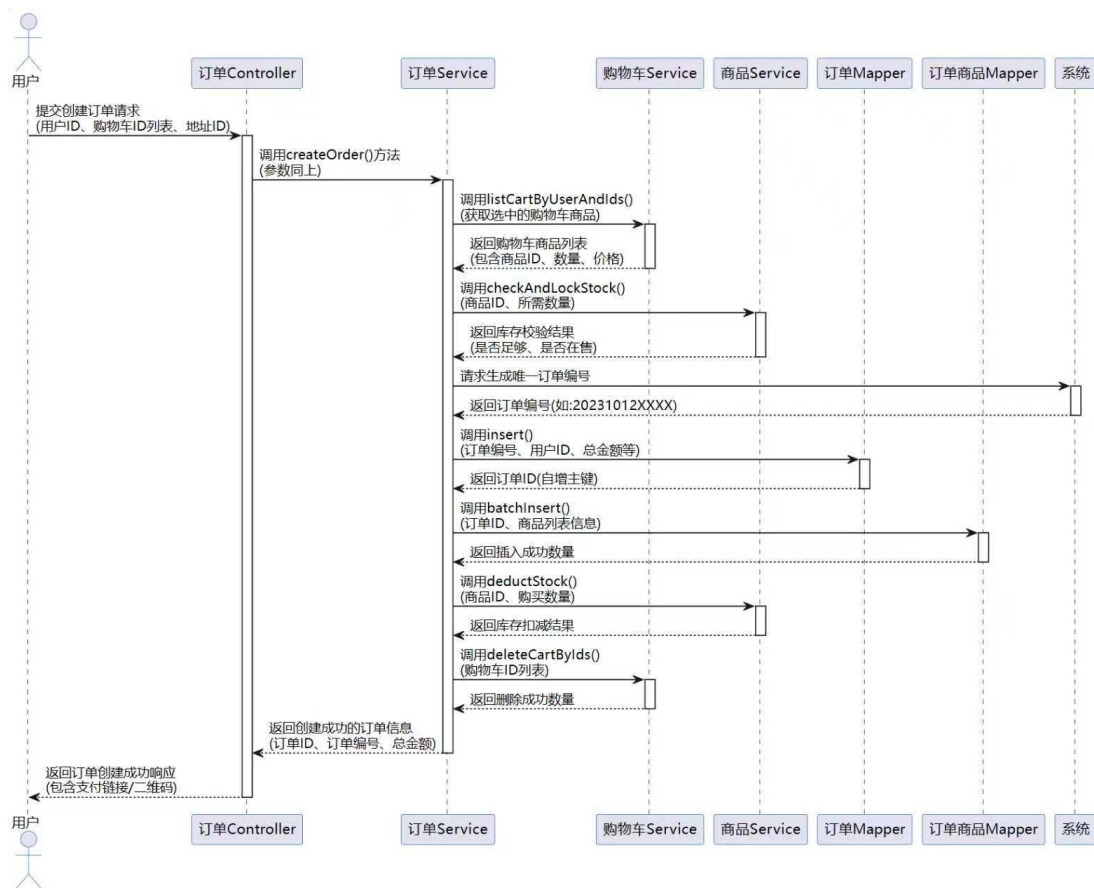


图 19 小组成员绘制顺序图

4.5.4 对象图

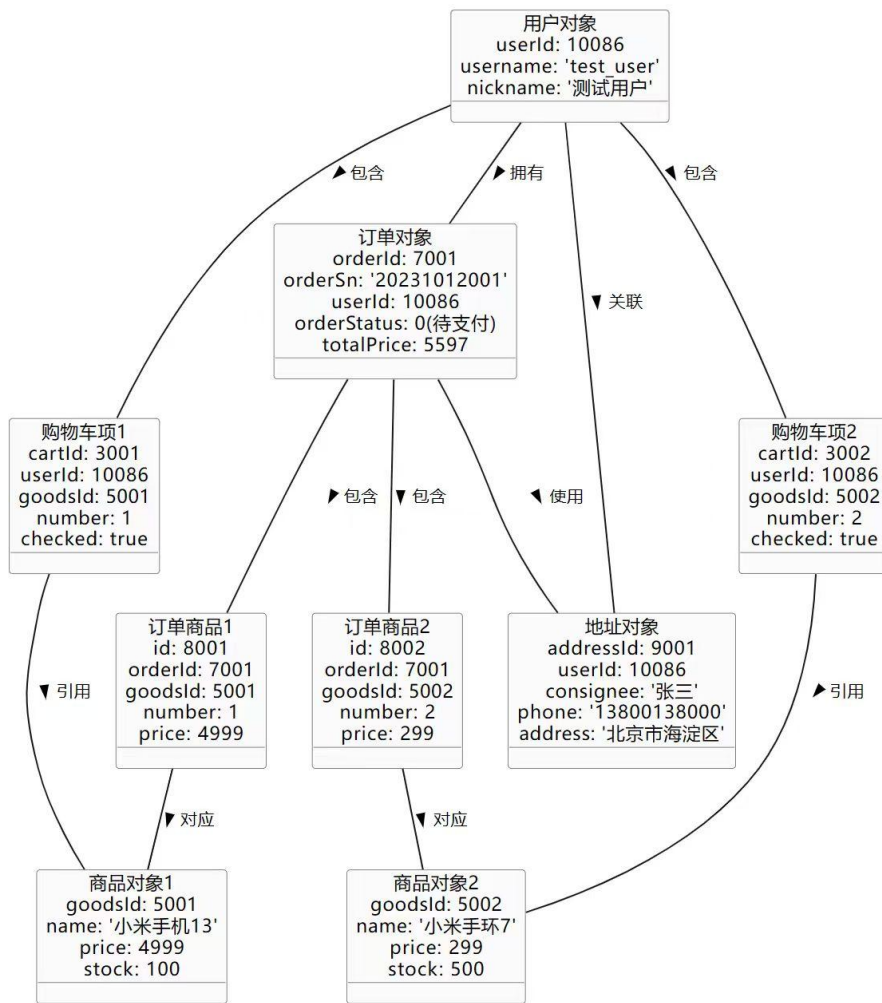


图 20 小组成员绘制对象图

4.5.5 组件图

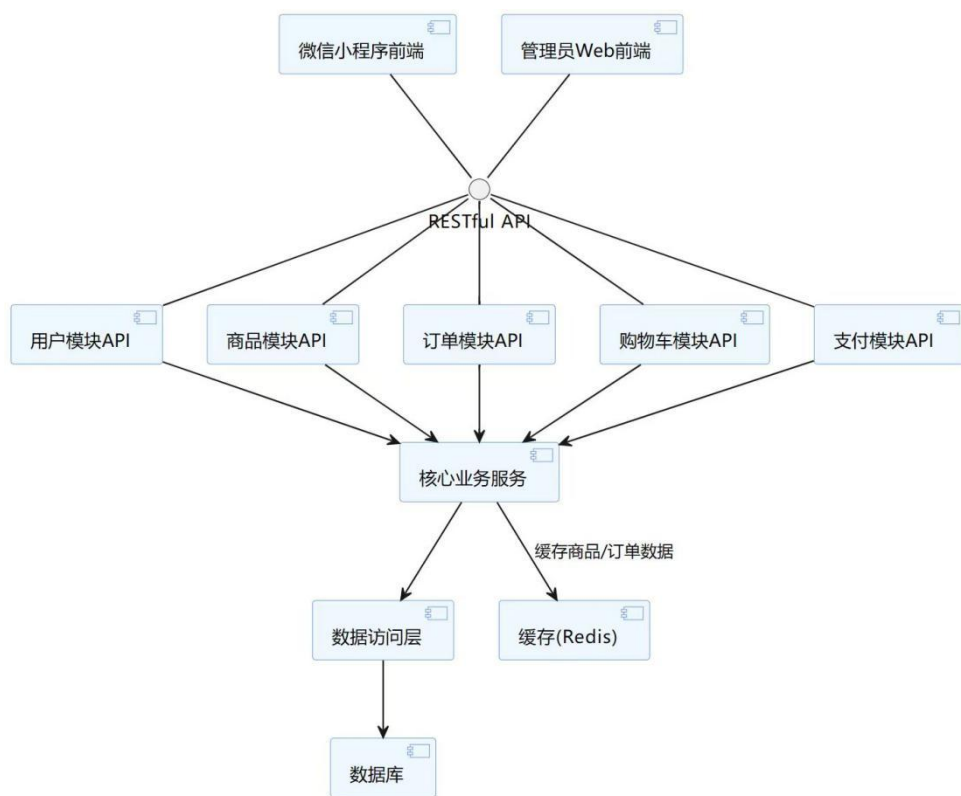


图 21 小组成员绘制组件图

第 5 章 会议记录

5.1 第一次会议记录

会议纪要

一、会议基本信息

1. 会议主题：讨论关于系统分析的选题问题
2. 时间：2025 年 9 月 27 日
3. 地点：27 教
4. 主持人：曹子墨
5. 参会人：曹子墨、崔胤康、韩方俊、胡睿杰、周厚泽
6. 记录人：韩方俊

二、会议核心内容

1. 议题讨论：

- 议题 1：是否从 0 开始进行新的项目
- 议题 2：对于选取项目的质疑

2. 决议事项：

- 决议 1：会议决定从 gitee 上选取项目 litemall 作为分析对象
- 决议 2：会议打消了质疑

三、后续安排

1. 待办任务：

- 任务 1：整理关于项目的拆解与介绍 曹子墨-韩方俊 2025.10.10
- 任务 2：整理面向对象等基础知识以备使用 崔胤康-周厚泽 2025.10.10
- 任务 3：整理用例模型与适用工具 胡睿杰 2025.10.9

3. 下次会议：2025 年 10 月 11 日

5.2 第一次会议记录

会议纪要

一、会议基本信息

1. 会议主题：讨论关于项目的初步研究与第一份报告草稿的撰写
2. 时间：2025 年 10 月 10 日
3. 地点：8 教
4. 主持人：曹子墨
5. 参会人：曹子墨、崔胤康、韩方俊、胡睿杰、周厚泽
6. 记录人：韩方俊

二、会议核心内容

1. 议题讨论：

- 议题 1：检查资料收集的问题
- 议题 2：讨论关于项目的研究程度

2. 决议事项：

- 决议 1：资料收集的信息来源应该更加完善，更加贴近我们所分析项目的情况
- 决议 2：每个人在 pull 项目之后都对项目进行了初步的解剖，是很好的事，应当随着项目的推进继续探究。

三、后续安排

1. 待办任务：

- 任务 1：将所有成员的调查结果整合并上传平台 2025.10.13

2. 下次会议：2025 年 10 月 17 日

参考文献

- [1] Sabah Al-Fedaghi, Classes in Object-Oriented Modeling (UML): Further Understanding and Abstraction, arXiv preprint, arXiv:2106.00267, 2021.
- [2] Peter Wegner, What is Object-Oriented Programming?, IEEE Software, vol. 1, no. 1, pp. 17-24, 1987.
- [3] Al-Fedaghi, S., Classes in Object-Oriented Modeling (UML): Further Understanding and Abstraction, 2021.
- [4] Wegner, P., What is Object-Oriented Programming?, IEEE Software, 1987.
- [5] Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., & Lorensen, W. Object-Oriented Modeling and Design. Prentice Hall, 1991.
- [6] Booch, G. Object-Oriented Analysis and Design with Applications. 3rd Edition, Addison-Wesley, 2007.
- [7] Wegner, P., What is Object-Oriented Programming?, IEEE Software, vol. 1, no. 1, pp. 17 - 24, 1987.
- [8] Object Management Group (OMG). Unified Modeling Language Specification (UML 2.5.1). OMG, 2017.
- [9] UML-Diagrams.org. UML Diagram Overview [EB/OL]. (2024-05-12).
- [10] GeeksforGeeks. UML Use Case Diagram [EB/OL]. (2024-05-12).
- [11] Wikipedia. Class Diagram [EB/OL]. (2024-05-12).
- [12] Wikipedia. Object Diagram [EB/OL]. (2024-05-12).
- [13] Visual Paradigm. Creating Sequence Diagrams with a Use Case-Driven Approach [EB/OL]. (2024-05-12).
- [14] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [15] Jacobson I. OOPSLA'87 Proceedings. 1987
- [16] Jacobson I, et al. Object-Oriented Software Engineering. Addison-Wesley, 1992.
- [17] OMG. UML 2.5 Specification. 2015
- [18] Ambler S W. Agile Modeling. Wiley, 2002.
- [19] Bittner K, Spence I. Use Case Modeling. Addison-Wesley, 2002.
- [20] Cockburn A. Writing Effective Use Cases. Addison-Wesley, 2000:57.
- [21] Armour F, Miller G. Advanced Use Case Modeling. Addison-Wesley, 2003.
- [22] Kulak D, Guiney E. Use Cases: Requirements in Context. 2nd ed. Addison-Wesley, 2004.
- [23] Larman C. Applying UML and Patterns. 3rd ed. Prentice Hall, 2004:128.
- [24] Fowler M. UML Distilled. 3rd ed. Addison-Wesley, 2003.
- [25] Rosenberg D. Use Case Driven Object Modeling. Apress, 2007.
- [26] Atlassian. Agile Requirements Management. 2020.
- [27] Sparx Systems. Enterprise Architect User Guide. 2021.
- [28] Wiegers K. Software Requirements. 3rd ed. Microsoft Press, 2013:79.
- [29] Adolph S, et al. Patterns for Effective Use Cases. Addison-Wesley, 2002.
- [30] Evans E. Domain-Driven Design. Addison-Wesley, 2003:24-26.

- [31]Rumbaugh J, Blaha M, Premerlani W, et al. Object-Oriented Modeling and Design[M]. Prentice Hall, 1991.
- [32]Larman C. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development[M]. 3rd ed. Prentice Hall, 2004.
- [33]Fowler M. UML Distilled: A Brief Guide to the Standard Object Modeling Language[M]. 3rd ed. Addison-Wesley, 2003.
- [34]George L, Herschmann L, Satzinger J. Object-Oriented Systems Analysis and Design[M]. Course Technology, 2007.
- [35] Wirfs-Brock R. 2002.
- [36]Bass L. Software Architecture in Practice. 3rd ed. Addison-Wesley, 2012.
- [37]Martin R. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- [38]Richardson C. Microservices Patterns: With Examples in Java. Manning Publications, 2018.
- [39]Bloch J. Effective Java. 3rd ed. Addison-Wesley, 2018.
- [40]Meyer B. Object-Oriented Software Construction. 2nd ed. Prentice Hall, 1997.
- [41]Freeman E. Head First Design Patterns. O'Reilly Media, 2004.
- [42]Fowler M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

